Software Architecture and Design Specification

Project: Online Complaint Management

Complaint Management System

Authors: Pallavi J, Prajna Holla, Puneeth Yenneti, Rakshita Muttur

Date: 3-10-2026

Status: Draft

Revision History

Revision History	Date	Author	Change Summary
1.0	1-10-2026	Rakshita Muttur	Initial SAD draft based on the SRS templates
1.1	6-10-2026	Rakshita Muttur	Included the API Design, UML diagrams.

Approvals

Role	Name	Signature/ Date
Project Representative		
Instructor/ Guide		

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Online Complaint Management System, a web application through which users lodge complaints and track their resolution, and administrators review, assign and resolve them.

1.2 Scope

Covers user registration and authentication, role-based access, complaint submission with unique ID generation, status tracking and history, administrative assignment and resolution, search and filtering, and status-change notifications (FR-01 to FR-13). Payment, mobile apps and external department integrations are out of scope.

1.3 Audience
Developers, QA engineers, security auditors, instructors, and maintenance teams.

1.4 Definitions
FR (Functional Requirement), NFR (Non-Functional Requirement), SAD (Software Architecture Document), ADR (Architecture Decision Record), RBAC (Role-Based Access Control), REST, JWT/Session, BCrypt, STRIDE, TLS, UML.

2. Document Overview

2.1 How to use this document
Provides the component diagram, architecture rationale, security architecture, sequence diagrams, API definitions and requirement traceability. Read Section 3 for structure and Section 4 for behaviour and interfaces.

2.2 Related Documents
SRS (FR-01 to FR-13, NFR-01 to NFR-10), STP (Software Test Plan), RTM (Requirements Traceability Matrix).

3. Architecture

3.1 Goals & Constraints

Goals: Secure handling of credentials and complaints; no loss of submitted complaints; responses within 3 s (NFR-01); at least 100 concurrent users (NFR-02); easy for non-technical users (NFR-05); modular and easy to extend with new complaint categories.

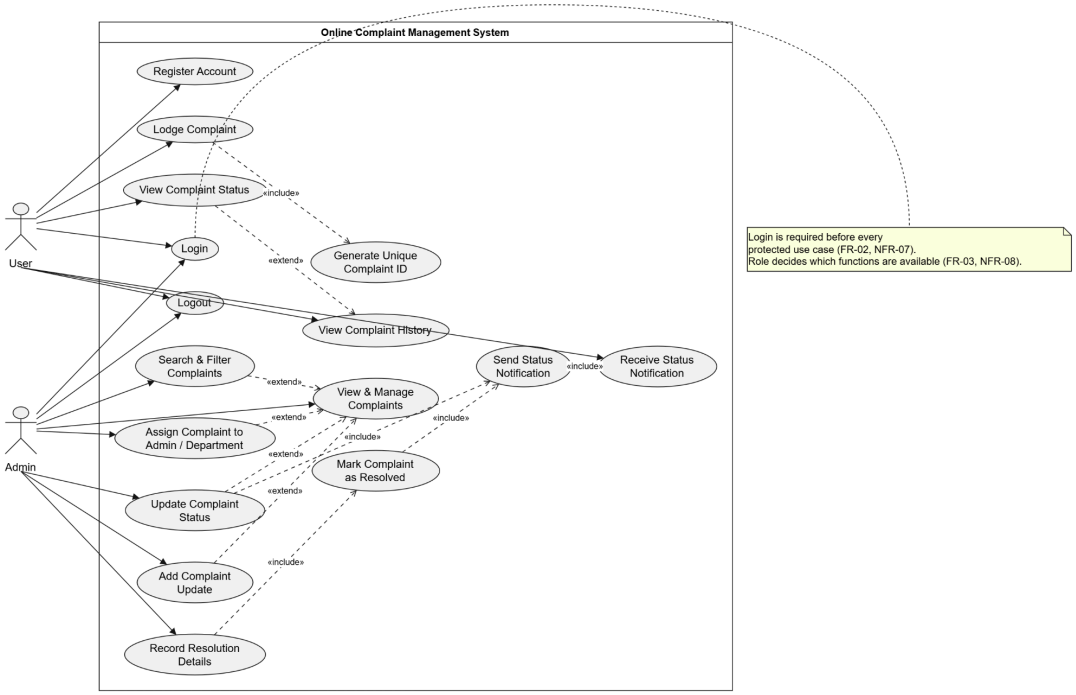
Constraints: Web-based interface only; relational database; academic mini-project scope (small team, limited time); passwords must be hashed, never stored in plaintext (NFR-09).

3.2 Stakeholders & Concerns

Users (complainants): ease of use, status visibility, privacy of their complaints

- **Administrators:** efficient triage, assignment, search and resolution
- **Responsible departments:** clear assignment and update trail
- **Developers:** modularity, testability
- **Instructors / reviewers:** traceability, documentation quality
- **Security auditors:** authentication, authorization, data protection

3.3 Component (UML) Diagram



3.4 Component Descriptions

- **Web UI:** registration, login, complaint form, status/history views, admin dashboard; shows validation and error messages.
- **Auth & Session Service:** registers users, verifies credentials, hashes passwords, enforces role-based access, creates and invalidates sessions.
- **Complaint Service:** validates and stores complaints, generates the unique complaint ID, returns submission confirmation, serves a user's own complaints, status and history.
- **Admin Management Service:** lets administrators view and manage complaints, assign them to an administrator or department, add updates, record resolution and change status.
- **Search & Filter Module:** queries complaints by ID, category, status and submission date (administrators only).
- **Notification Service:** records and delivers in-app (optionally email) notifications on significant status changes.
- **Repository / DAO:** all database access, with transactions for atomic writes.
- **MySQL Database:** Users, Roles, Complaints, Complaint_History, Departments, Notifications.

3.5 Chosen Architecture Pattern and Rationale

A **layered (3-tier) architecture**: Presentation, Application and Data layers. It gives clear separation of concerns, maps directly to the FR groups, is easy to test, and fits the project's scale. **Microservices were rejected** as overly complex for about 100 concurrent users and a small team, adding deployment and network overhead with no benefit. A **monolithic deployment with modular internal services** keeps a future split possible.

3.6 Technology Stack & Data Stores

- **Frontend:** Django Templates with HTML5, CSS, JavaScript (Bootstrap optional for responsive pages)
- **Backend:** Python, Django (MVT architecture), with Django's built-in authentication and permissions for role-based access (Django REST Framework optional if you expose JSON APIs)
- **Database:** MySQL (relational, ACID transactions to protect complaint data), accessed through Django ORM
- **Security:** HTTPS/TLS, Django's built-in PBKDF2 password hashing, server-side sessions (Django session framework), CSRF protection.

3.7 Risks & Mitigations

Risk	Mitigation
Failed submission creates duplicate complaints (NFR-04)	Atomic transaction; ID from a DB sequence; disable the submit button after click; return a clear error on failure
Slow responses under load (NFR-01/02)	Indexed columns (ID, status, category, date); connection pooling; pagination of lists
Data loss (NFR-03)	Transactional commits; regular database backups
Credential theft	BCrypt hashing; no plaintext storage or logging of passwords
Unauthorized access to admin functions	Server-side role checks on every protected endpoint
Session misuse after logout (NFR-10)	Invalidate the server session on logout and set short timeouts
Missed notifications	Store each notification in the DB so it appears in-app even if email fails

3.8 Traceability to Requirements

FR-01 User Registration	Auth Service, Web UI

FR-02 User Authentication; NFR-07	Auth Service
FR-03 Role-Based Access; NFR-08	Auth Service (RBAC), all controllers
FR-04 Lodge Complaint; FR-05 ID Generation	Complaint Service, Repository
FR-06 View Status; FR-07 History	Complaint Service, Complaint_History table
FR-08 Complaint Update	Complaint Service / Admin Service
FR-09 Admin Complaint Management	Admin Service
FR-10 Assignment; FR-11 Resolution	Admin Service
FR-12 Search and Filtering	Search & Filter Module
FR-13 Notifications	Notification Service
NFR-01, NFR-02 Performance	Indexing, connection pooling, pagination (all layers)
NFR-03 Persistence; NFR-04 Error Recovery	Repository (transactions), global error handler
NFR-05, NFR-06 Usability, Validation	Web UI, server-side validation
NFR-09 Password Protection	Auth Service (BCrypt)
NFR-10 Session Security	Auth Service (logout invalidation)

3.9 Security Architecture

Security objectives: SO-01 (authentication and access control) and SO-02 (data protection).

Threat modeling (STRIDE):

- **Spoofing:** credential authentication, hashed passwords, session tokens
- **Tampering:** server-side validation, parameterized queries (prevents SQL injection), role checks before any update
- **Repudiation:** Complaint_History logs every status change and update with actor and timestamp
- **Information disclosure:** TLS in transit; users see only their own complaints; no sensitive data in logs or error messages
- **Denial of service:** rate limiting on login and submission; pagination
- **Elevation of privilege:** RBAC enforced server-side; admin endpoints reject non-admin sessions (NFR-08)

4. Design

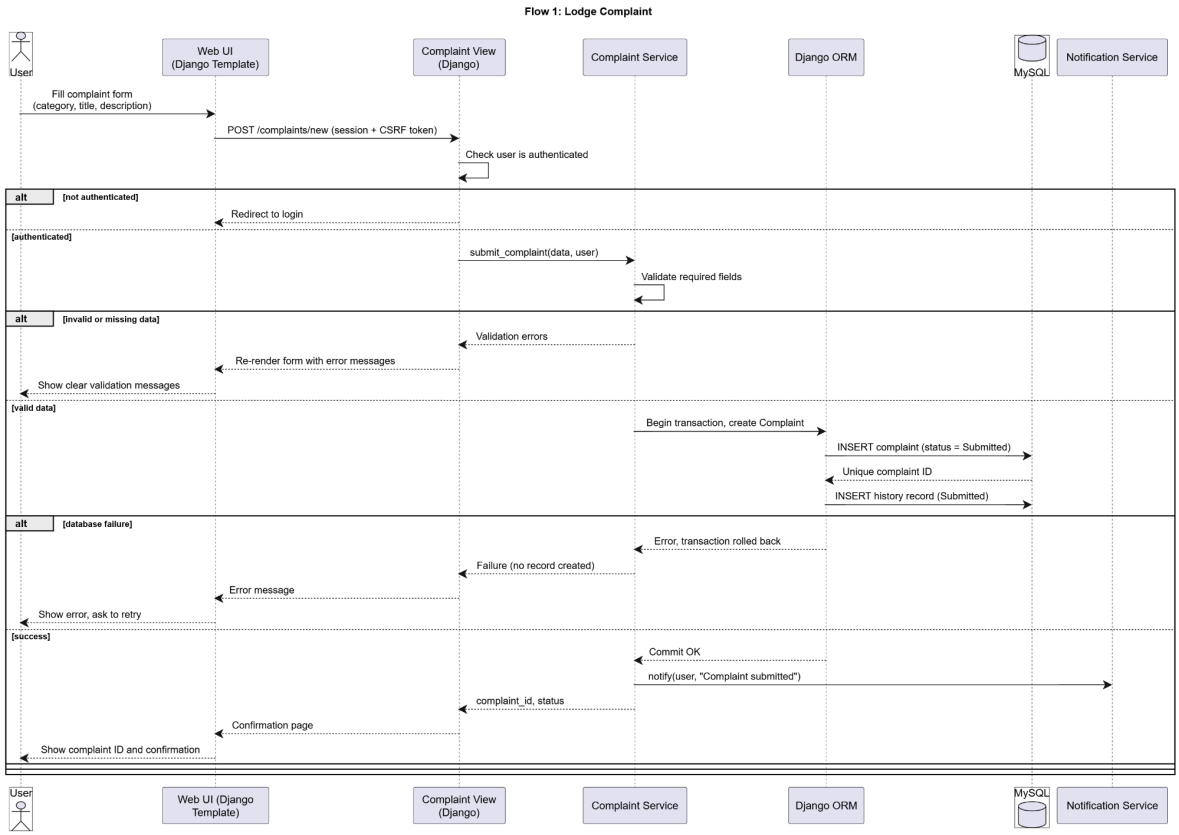
4.1 Design Overview

The system is organised into layered modules. Controllers handle HTTP requests and validation, services hold business rules (status workflow, ID generation, assignment), and repositories handle persistence. Complaint status follows the workflow **Submitted → Under Review → Assigned → In Progress → Resolved**, with **Rejected / Closed** as additional terminal states. Every transition writes a Complaint_History record and triggers a notification

4.2 UML Sequence Diagrams

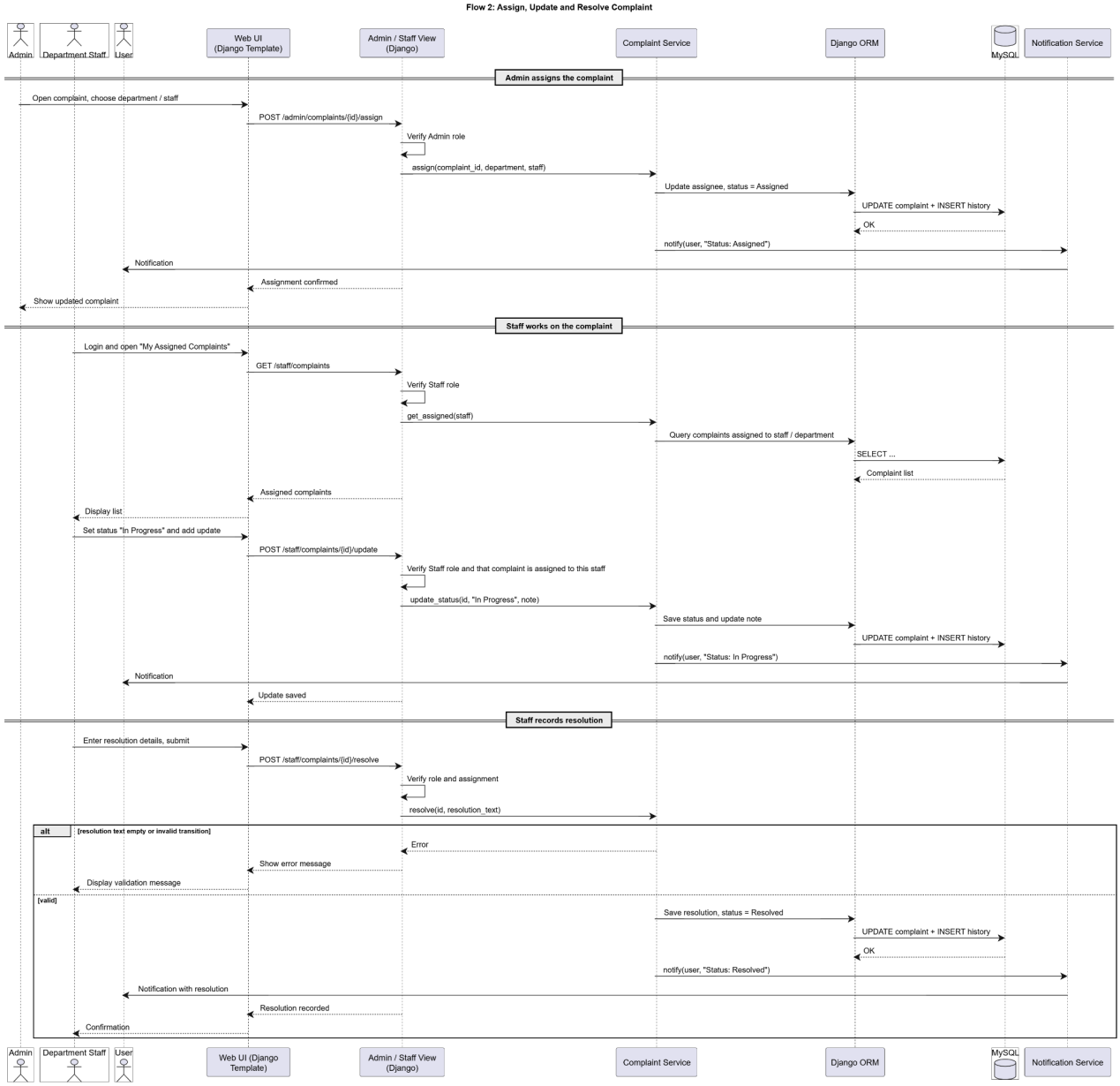
Flow 1 - Lodging a complaint

This flow covers the FR-04/FR-05 happy path plus the NFR-04 failure path (rollback, no duplicate record) and NFR-06 validation feedback.



Flow 2 - Assign, Update and Resolve Complaint

This flow shows the three-actor workflow: Admin assigns, Staff updates and resolves, and the User is notified at each status change (FR-13). Role checks appear as explicit steps (NFR-08), and every change writes a history record (FR-07).



4.3 API Design

A. Authentication and Account Management (FR-01, FR-02, FR-03, NFR-09, NFR-10)

1. Register
    - Endpoint: POST /register
    - Access: Public
    - Request: name, email, password, confirm_password
    - Success: 201 with user_id and a confirmation message
    - Errors: 400 for invalid or missing fields; 409 if the email is already registered
2. Login
    - Endpoint: POST /login
    - Access: Public
    - Request: email, password
    - Success: 200 with the user's role and a message; a session is created
    - Errors: 400 for missing fields; 401 for invalid credentials
3. Logout
    - Endpoint: POST /logout
    - Access: Authenticated users
    - Request: session
    - Success: 200 with a message; the session is invalidated
    - Errors: 401 if not authenticated

B. Complaint Submission and Tracking: User (FR-04 to FR-07, NFR-03 to NFR-06)

1. Lodge Complaint
    - Endpoint: POST /complaints/new
    - Access: User
    - Request: category, title, description
    - Success: 201 with complaint_id, status (Submitted) and a confirmation message
    - Errors: 400 with field-wise validation messages; 401 if not authenticated; 500 if saving fails (no record is created)
2. View My Complaints
    - Endpoint: GET /complaints
    - Access: User
    - Request: none
    - Success: 200 with the list of the user's own complaints (complaint_id, title, category, status, submitted date)
    - Errors: 401 if not authenticated
3. View Complaint Details and History
    - Endpoint: GET /complaints/{id}
    - Access: User (owner of the complaint)

- Request: none
- Success: 200 with complaint details, current status, resolution (if any) and status history
- Errors: 401; 403 if the complaint belongs to another user; 404 if not found

4. View Notifications
   - Endpoint: GET /notifications
   - Access: User
   - Request: none
   - Success: 200 with the list of notifications (message, complaint_id, timestamp)
   - Errors: 401 if not authenticated

C. Administrative Management (FR-08 to FR-13, NFR-08)

1. List, Search and Filter Complaints
   - Endpoint: GET /admin/complaints
   - Access: Admin
   - Request: optional filters complaint_id, category, status, from_date, to_date, and page
   - Success: 200 with a paginated list of complaints
   - Errors: 401; 403 if the user is not an admin
2. View Complaint (Admin)
   - Endpoint: GET /admin/complaints/{id}
   - Access: Admin
   - Request: none
   - Success: 200 with full complaint details and history
   - Errors: 401; 403; 404
3. Assign Complaint
   - Endpoint: POST /admin/complaints/{id}/assign
   - Access: Admin
   - Request: assignee or department
   - Success: 200 with status Assigned; the user is notified
   - Errors: 400 for an invalid assignee; 403; 404; 409 for an invalid status transition
4. Add Update / Change Status
   - Endpoint: POST /admin/complaints/{id}/update
   - Access: Admin
   - Request: update_note, optional status
   - Success: 200 with the new status and history entry; the user is notified on a status change
   - Errors: 400 for an empty note or invalid status; 403; 404
5. Resolve Complaint
   - Endpoint: POST /admin/complaints/{id}/resolve
   - Access: Admin

- Request: resolution_text
- Success: 200 with status Resolved; the user is notified
- Errors: 400 for an empty resolution; 403; 404; 409 if already resolved or the transition is invalid

D. Common Conventions

- Authentication: A Django session is created at login and destroyed at logout.
- Authorization: All /admin routes reject non-admin users with a 403 response. Users can access only their own complaints.
- Error format: Each error response contains a status code, a readable message and, for validation failures, a field-wise list of reasons (NFR-06).
- Status workflow: Submitted → Under Review → Assigned → In Progress → Resolved, with Rejected/Closed as additional states. Requests that skip or reverse a state return 409.
- Duplicate prevention (NFR-04): Complaint creation runs in a single database transaction, so a failed request leaves no record and a retry cannot create a duplicate.
- Performance (NFR-01): List endpoints are paginated and the searched columns are indexed.

4.4 Error Handling, Logging & Monitoring

- Standardized JSON error format {timestamp, status, message}; clear validation messages for missing or invalid fields (NFR-06) and friendly failure messages (NFR-04).
- A failed submission rolls back, so no partial or duplicate record is created.
- Logs capture request IDs, actions and failures. Passwords, session tokens and complaint text are never logged.
- Monitoring covers response times against the 3 s target, submission failure rate, login failure counts and database availability.

4.5 UX Design

Simple, responsive pages with a clear flow (register, login, lodge complaint, track). A status progress indicator shows Submitted through Resolved, and the complaint ID is shown prominently on confirmation. Forms show inline validation, and the admin dashboard has filter controls and a status badge on every row. Plain language is used throughout so no technical knowledge is needed (NFR-05).

4.6 Open Issues & Next Steps

- Confirm the exact concurrent-user target (NFR-02 is tentative) and test environment.
- Decide whether notifications are in-app only or also email.
- Finalize complaint categories and department list.
- Future enhancements: file attachments, SLA/escalation timers, analytics dashboard, multi-language support.

5. Appendices

5.1 Glossary: FR, NFR, SAD, ADR, RBAC, STRIDE, TLS, REST, BCrypt, Complaint ID, Status workflow.

5.2 References: IEEE 42010, OWASP Top 10, NIST SP 800-160.

5.3 Tools: PlantUML, draw.io, Swagger/Postman, MySQL Workbench, Git.